PRÁCTICA AUTENTICACIÓN Y CONTROL DE ACCESO EN SYMFONY

IES Alixar. 2º DAW. Curso 2025/26

ÍNDICE

1. Instalación de Symfony **1**
2. Implantación del servidor web **6**
2.1. Creación de un contenedor con el servidor web 6
2.2. Despliegue del contenedor web con Docker Compose 10
3. Implantación del servidor de BD **11**
3.1. Despliegue del contenedor de BD con Docker Compose 11
3.2. Contenedor para la gestión web del servidor de BD 13
3.3. Instalación y configuración de Doctrine 14
4. Seguridad: Usuarios y roles **16**
4.1. Requisitos previos 16
4.2. Creación de la entidad Usuario 17
4.3. Registro de usuarios 17
4.4. Formulario de login 18
4.5. Habilitar FormLoginAuthenticator 18
4.6. Control de acceso 18
5. Creación de un CRUD **19**
5.1. Dashboard de administración 19
5.2. Controladores CRUD 21
5.3. Visualización de campos de los CRUDs 23
5.4. Filtros en los CRUDs 24
5.5. Hashing de la contraseña en el CRUD de usuarios 24
5.6. Restringir el acceso al backend de administración 26

1. Instalación de Symfony

El framework Symfony está formado por un conjunto de archivos escritos en PHP, de modo que la instalación consiste en obtener esos archivos para que estén disponibles para los proyectos.

Antes de instalar Symfony es preciso instalar una serie de requerimientos:

- ➔ PHP 8.2 o superior (sudo apt install php) junto con las siguientes extensiones que suelen estar instaladas y habilitadas por defecto: Ctype, iconv, PCRE, Session, SimpleXML (`sudo apt install php-xml`), y Tokenizer;
- ➔ Instalar Composer, que se utiliza para la gestión de paquetes PHP.

```
php -r "copy('https://getcomposer.org/installer', 'composer-setup.php');"
php -r "if (hash_file('sha384', 'composer-setup.php') ===
'ed0feb545ba87161262f2d45a633e34f591ebb3381f2e0063c345ebea4d228dd0043083717
```

```
770234ec00c5a9f9593792') { echo 'Installer verified'.PHP_EOL; } else { echo
'Installer corrupt'.PHP_EOL; unlink('composer-setup.php'); exit(1); }"
php composer-setup.php
php -r "unlink('composer-setup.php');"
```

```
sudo mv composer.phar /usr/local/bin/composer
```

```
mlasmes@lubuntu-mlasmes:~$ php composer-setup.php
All settings correct for using Composer
Downloading...

Composer (version 2.8.12) successfully installed to: /home/mlasmes/com
poser.phar
Use it: php composer.phar
```

```
mlasmes@lubuntu-mlasmes:~$ sudo mv composer.phar /usr/local/bin/composer
[sudo] contraseña para mlasmes:
mlasmes@lubuntu-mlasmes:~$ composer -V
Composer version 2.8.12 2025-09-19 13:41:59
PHP version 8.3.6 (/usr/bin/php8.3)
Run the "diagnose" command to get more detailed diagnostics output.
mlasmes@lubuntu-mlasmes:~$ which composer
/usr/local/bin/composer
mlasmes@lubuntu-mlasmes:~$
```

La instalación de un proyecto Symfony se puede realizar a través de Composer, usando el siguiente comando:

```
composer create-project symfony/skeleton:"7.3.x" <nombre_proyecto>
cd <nombre_proyecto>
composer require webapp
```

No obstante, alternativamente vamos a utilizar el binario Symfony CLI para crear un proyecto con todos los archivos del framework Symfony. Instala Symfony CLI en Ubuntu mediante el gestor de paquetes APT:

```
curl -1sLf 'https://dl.cloudsmith.io/public/symfony/stable/setup.deb.sh' |
sudo -E bash
sudo apt install symfony-cli
```

El binario de symfony proporciona una herramienta para comprobar si el sistema cumple los requisitos necesarios para la creación de un proyecto de symfony.

```
symfony check:requirements
```

```
> Checking Symfony requirements:

..........W...W.........W

 [OK]
 Your system is ready to run Symfony projects
```

Antes de crear el proyecto es preciso establecer la identidad del usuario de git indicando su nombre y su correo electrónico. Esta información es importante porque se introduce de manera inmutable en los commits ejecutados por el usuario:

```
* Setting up the project under Git version control
  (running git init /home/mlasmes/pedidos)

Identidad del autor desconocido

*** Por favor cuéntame quién eres.

Ejecuta

  git config --global user.email "you@example.com"
  git config --global user.name "Tu Nombre"
```

```
git config --global user.name "Mario Lasso"
git config --global user.email "mlasmes459@g.educaand.es"ls -l
```

Crea un proyecto para una aplicación web tradicional:

```
symfony new --webapp <nombre_proyecto>
```

Por defecto, el proyecto se crea con una jerarquía de directorios. Cada tipo de componente debe estar en un directorio concreto. En el directorio raíz del proyecto se encuentra el fichero oculto .env (ls -la) que guarda la configuración de BD y correo.

```
mlasmes@lubuntu-mlasmes:~$ symfony new --webapp pedidos
* Creating a new Symfony project with Composer
* Setting up the project under Git version control
  (running git init /home/mlasmes/pedidos)

 [OK] Your project is now ready in /home/mlasmes/pedidos
```

```
mlasmes@lubuntu-mlasmes:~$ ls -l pedidos/
total 440
drwxrwxr-x  5 mlasmes mlasmes   4096 sep 20 14:02 assets
drwxrwxr-x  2 mlasmes mlasmes   4096 sep 20 14:02 bin
-rw-rw-r--  1 mlasmes mlasmes    331 sep 20 14:02 compose.override.yam
l
-rw-rw-r--  1 mlasmes mlasmes   3245 sep 20 14:02 composer.json
-rw-rw-r--  1 mlasmes mlasmes 369349 sep 20 14:02 composer.lock
-rw-rw-r--  1 mlasmes mlasmes    884 sep 20 14:02 compose.yaml
drwxrwxr-x  4 mlasmes mlasmes   4096 sep 20 14:00 config
-rw-rw-r--  1 mlasmes mlasmes    782 sep 20 14:02 importmap.php
drwxrwxr-x  2 mlasmes mlasmes   4096 sep 20 14:02 migrations
-rw-rw-r--  1 mlasmes mlasmes   1410 sep 20 14:02 phpunit.dist.xml
drwxrwxr-x  2 mlasmes mlasmes   4096 sep 20 14:00 public
drwxrwxr-x  5 mlasmes mlasmes   4096 sep 20 14:02 src
-rw-rw-r--  1 mlasmes mlasmes   9308 sep 20 14:02 symfony.lock
drwxrwxr-x  2 mlasmes mlasmes   4096 sep 20 14:02 templates
drwxrwxr-x  2 mlasmes mlasmes   4096 sep 20 14:02 tests
drwxrwxr-x  2 mlasmes mlasmes   4096 sep 20 14:02 translations
drwxrwxrwx  4 mlasmes mlasmes   4096 sep 20 14:02 var
drwxrwxr-x 21 mlasmes mlasmes   4096 sep 20 14:01 vendor
mlasmes@lubuntu-mlasmes:~$
```

Los directorios más importantes del proyecto son:

- `bin/`: El directorio bin/ contiene el punto de entrada principal a los comandos que se introducen desde la línea de comandos (console).
- `config/`: Está formado por un conjunto de archivos de configuración con valores predeterminados. En config/packages/ se encuentra el fichero security.yaml con la configuración de seguridad.
- `public/`: Es el directorio raíz de la web, y el script index.php es el principal punto de entrada para todos los recursos HTTP dinámicos.
- `src/`: En este directorio se guardan los ficheros con todo el código que escribe el desarrollador. Por defecto, todas las clases bajo este directorio usan el espacio de nombres de PHP App.
    - `src/Controller`: Para los controladores.
    - `src/Entity`: Guarda las entidades.
    - `src/Templates`: Para las plantillas.
- `var/`: El directorio var/ contiene cachés, logs y archivos generados en tiempo de ejecución por la aplicación. Necesita permisos de escritura en producción.
- `vendor/`: contiene todos los paquetes instalados por Composer, incluyendo el propio Symfony. Este directorio no debe modificarse.

Ejecuta el siguiente comando dentro del directorio del proyecto para mostrar información sobre el proyecto: `php bin/console about`

```
mlasmes@lubuntu-mlasmes:~/pedidos$ php bin/console abou
 -------------------- ---------------------------------
  Symfony
 -------------------- ---------------------------------
  Version              7.3.3
  Long-Term Support    No
  End of maintenance   01/2026 (in +133 days)
  End of life          01/2026 (in +133 days)
 -------------------- ---------------------------------
  Kernel
 -------------------- ---------------------------------
  Type                 App\Kernel
  Environment          dev
  Debug                true
  Charset              UTF-8
  Cache directory      ./var/cache/dev (8.8 MiB)
  Build directory      ./var/cache/dev (8.8 MiB)
  Log directory        ./var/log (5 KiB)
 -------------------- ---------------------------------
  PHP
 -------------------- ---------------------------------
  Version              8.3.6
  Architecture         64 bits
  Intl locale          n/a
  Timezone             UTC (2025-09-20T12:09:56+00:00)
  OPcache              Enabled
  APCu                 Not installed
  Xdebug               Not installed
 -------------------- ---------------------------------
```

2. Implantación del servidor web

2.1. Creación de un contenedor con el servidor web

Vamos a utilizar el siguiente `Dockerfile` disponible en el siguiente artículo, y lo vamos a ubicar en el directorio raíz del proyecto. Este Dockerfile utiliza la imagen base oficial php con tag 8.4-apache que incluye un servidor web Apache httpd sobre Debian con el módulo mod_php PHP 8.4 para la interpretación del código en el lado del servidor, el cual es compatible con el framework Symfony 7. El hecho de utilizar la última versión de PHP y del framework reduce el número de vulnerabilidades que afectan a la aplicación web.

En primer lugar, descargamos la imagen base mediante el comando `docker pull`. Este paso no es necesario, ya que la imagen se descarga automáticamente al crear el contenedor si esta no se encuentra localmente, pero la descarga manual nos sirve para comprobar que está disponible en Docker Hub.

```
mlasmes@lubuntu-mlasmes:~$ docker pull php:8.4-apache
8.4-apache: Pulling from library/php
ce1261c6d567: Pull complete
e35f4ad0b9dd: Pull complete
1b9a47e0f0aa: Pull complete
71717529431a: Pull complete
f861da3588da: Pull complete
0c0b33aa5bdb: Pull complete
4c32d2c29006: Pull complete
3b0476ed41ac: Pull complete
528d7cab555c: Pull complete
83cf044a4089: Pull complete
f54b8dc4cf72: Pull complete
d2dc8b9fbab9: Pull complete
03b4ff87f3a5: Pull complete
0e25f44ad642: Pull complete
4f4fb700ef54: Pull complete
Digest: sha256:2c308cf745592a8d17e1dd0fd87d44be472d23be081ec5c
55bc89843
Status: Downloaded newer image for php:8.4-apache
docker.io/library/php:8.4-apache
mlasmes@lubuntu-mlasmes:~$ docker images
REPOSITORY    TAG          IMAGE ID       CREATED       SIZE
php           8.4-apache   d04f3fc12f4c   2 weeks ago   511MB
hello-world   latest       1b44b5a3e06a   5 weeks ago   10.1kB
mlasmes@lubuntu-mlasmes:~$
```

Podemos probar la imagen creando un contenedor de prueba a partir de ésta. Para ello, ejecutamos el comando `docker run --rm --name webserver -p 80:80 -d php:8.4-apache`.

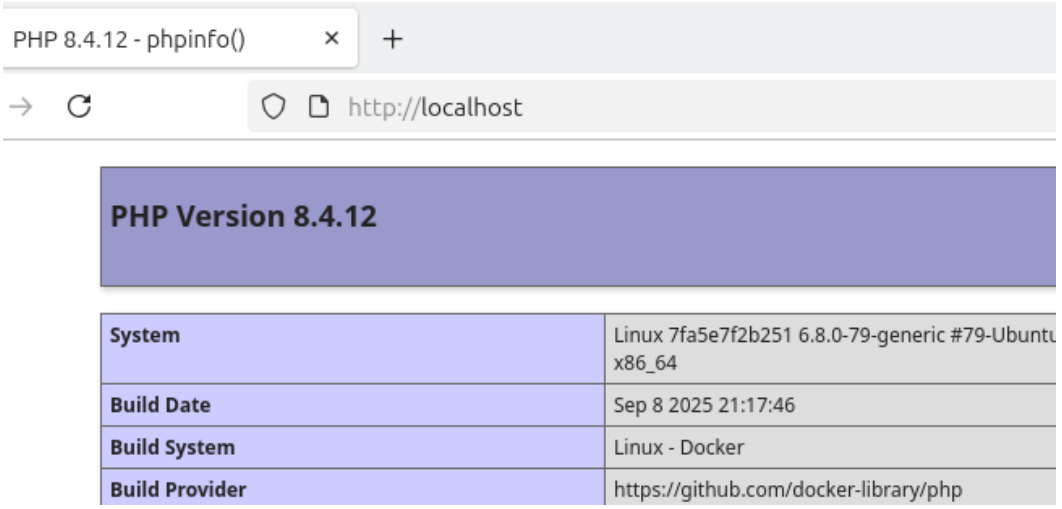
- La opción `--rm` elimina el contenedor cuando éste se para.
- La opción `-d` ejecuta el contenedor en background.
- La opción `-p` mapea un puerto del host en el puerto interno indicado del contenedor.

```
mlasmes@lubuntu-mlasmes:~$ docker run --rm --name webserver -p 80:80 -
d php:8.4-apache
7fa5e7f2b2516f192fd9470fc3992c78f58be3d69b355b33b28132887ac901c5
mlasmes@lubuntu-mlasmes:~$ docker ps
CONTAINER ID   IMAGE            COMMAND                  CREATED
   STATUS         PORTS                                   NAMES
7fa5e7f2b251   php:8.4-apache   "docker-php-entrypoi…"   3 seconds ago
   Up 2 seconds   0.0.0.0:80->80/tcp, [::]:80->80/tcp    webserver
```

Copiamos en el contenedor una página `index.php` con código PHP de prueba y abrimos una shell para ver que la página está en la ruta especificada. Tras esto, comprobamos que tenemos acceso desde el navegador en `http://localhost`.

```
mlasmes@lubuntu-mlasmes:~$ cat index.php
<?php
echo phpinfo();
?>
mlasmes@lubuntu-mlasmes:~$
```

```
mlasmes@lubuntu-mlasmes:~$ docker cp index.php webserver:/var/www/html
Successfully copied 2.05kB to webserver:/var/www/html
mlasmes@lubuntu-mlasmes:~$ docker exec -it webserver bash
root@7fa5e7f2b251:/var/www/html# ls
index.php
root@7fa5e7f2b251:/var/www/html#
```



Para la ejecución del proyecto Symfony será preciso instalar una serie de paquetes en el sistema Debian de la imagen:

```
RUN apt-get update && apt-get install -y \
    git \
    curl \
    vim \
    unzip \
    && rm -rf /var/lib/apt/lists/*
```

Del mismo modo, en el Dockerfile añadimos los pasos necesarios para instalar las dependencias de PHP con Composer y darle los permisos necesarios al usuario de www-data de apache:

```
RUN curl -sS https://getcomposer.org/installer | php \
    && php composer.phar install \
    && chown -R www-data:www-data /var/www/html \
    && chmod -R 777 /var/www/html/var
```

Adicionalmente, la documentación oficial recomienda añadir a la imagen un fichero de configuración php.ini personalizado. La imagen incluye un fichero php.ini-development y otro php.ini-production de ejemplo, más adecuado para entornos de producción.

```
# Use the default production configuration
RUN mv "$PHP_INI_DIR/php.ini-production" "$PHP_INI_DIR/php.ini"
```

En la imagen, el directorio raíz web de Apache por defecto está en `/var/www/html`. Según la documentación, esta ruta se puede cambiar estableciendo la variable de entorno APACHE_DOCUMENT_ROOT y modificando las líneas correspondientes en los ficheros de configuración global y del sitio por defecto. Establecemos el directorio raíz web de Apache en el directorio público del proyecto (`/var/www/html/public`):

```
RUN sed -ri -e 's!/var/www/html!${APACHE_DOCUMENT_ROOT}!g'
/etc/apache2/sites-available/*.conf
```

```
RUN sed -ri -e 's!/var/www/!${APACHE_DOCUMENT_ROOT}!g'
/etc/apache2/apache2.conf /etc/apache2/conf-available/*.conf
ENV APACHE_DOCUMENT_ROOT /var/www/html/public
```

Construimos la imagen personalizada a partir de la imagen base anterior ejecutando el comando docker build en el directorio del proyecto donde se encuentra el fichero Dockerfile e indicamos un nombre y una etiqueta para la nueva imagen:

```
docker build -t webserver:1.0 .
```

```
mlasmes@lubuntu-mlasmes:~/pedidos$ docker build -t webserver:1.0 .
[+] Building 65.4s (14/14) FINISHED                          docker:default
 => [internal] load build definition from Dockerfile                    0.0s
 => => transferring dockerfile: 1.30kB                                  0.0s
 => WARN: LegacyKeyValueFormat: "ENV key=value" should be used instead  0.0s
 => [internal] load metadata for docker.io/library/php:8.4-apache       0.0s
 => [internal] load .dockerignore                                       0.0s
 => => transferring context: 2B                                         0.0s
 => [1/9] FROM docker.io/library/php:8.4-apache                         0.5s
 => [internal] load build context                                       2.3s
 => => transferring context: 61.40MB                                    2.2s
 => [2/9] RUN apt-get update && apt-get install -y     git     curl     8.4s
 => [3/9] RUN docker-php-ext-install pdo pdo_mysql                      9.6s
 => [4/9] RUN a2enmod rewrite                                           0.3s
 => [5/9] RUN sed -ri -e 's!/var/www/html!${APACHE_DOCUMENT_ROOT}!g' /e 0.2s
 => [6/9] RUN sed -ri -e 's!/var/www/!${APACHE_DOCUMENT_ROOT}!g' /etc/a 0.2s
 => [7/9] COPY . /var/www/html/                                         1.3s
 => [8/9] WORKDIR /var/www/html/                                        0.1s
 => [9/9] RUN curl -sS https://getcomposer.org/installer | php     &&  42.5s
 => exporting to image                                                  2.1s
 => => exporting layers                                                 2.1s
 => => writing image sha256:e811950052df64e256112ea2826b621d47d12c5b126 0.0s
 => => naming to docker.io/library/webserver:1.0                        0.0s

 1 warning found (use docker --debug to expand):
 - LegacyKeyValueFormat: "ENV key=value" should be used instead of legacy "ENV
 key value" format (line 21)
```

Creamos un contenedor a partir de la nueva imagen escuchando en el puerto TCP 80:

```
docker run --rm --name webserver -p 80:80 -d webserver:1.0
```

Recuerda que es preciso que el puerto TCP 80 esté libre (`netstat -tln`) para poder levantar el contenedor. Si ya existe un servidor web escuchando en dicho puerto, será preciso pararlo (`sudo systemctl stop apache2`) y deshabilitarlo (`sudo systemctl disable apache2`) en el arranque del sistema para evitar conflictos.

```
mlasmes@lubuntu-mlasmes:~/pedidos$ docker run --rm --name webserver -p 80:80 -
d webserver:1.0
8bfe07a256a22658c0b8d0043f6406d795e89e79090d2cd218c966b4c36ade90
mlasmes@lubuntu-mlasmes:~/pedidos$ docker ps
CONTAINER ID   IMAGE           COMMAND                  CREATED         STATUS
         PORTS                                     NAMES
8bfe07a256a2   webserver:1.0   "docker-php-entrypoi…"   3 seconds ago   Up 2 s
econds   0.0.0.0:80->80/tcp, [::]:80->80/tcp   webserver
mlasmes@lubuntu-mlasmes:~/pedidos$
```

Tras esto, paramos el contenedor `docker stop webserver`, de modo que se eliminará debido a la opción `-rm` que fue usada en el comando `docker run`.

2.2. Despliegue del contenedor web con Docker Compose

En primer lugar, mueve a otra ubicación los ficheros compose.yaml y compose.override.yaml que trae el proyecto por defecto para la creación de un contenedor PostgreSQL, ya que vamos a definir nuestro propio fichero yaml.

Define una primera versión de un fichero `compose.yaml` en el raíz del proyecto, el cual va a servir para ejecutar de forma más sencilla la imagen Docker que hemos creado con el código del proyecto. Docker Compose es una herramienta para crear y ejecutar aplicaciones multi-contenedor. Posteriormente, añadiremos otro contenedor en la misma red interna que ejecute el servidor de BD al que se conectará la aplicación.

```
services:
  # Define el servicio web
  web:
    build: .
    container_name: webserver
    volumes:
      - .:/var/www/html
    ports:
      - "80:80"
    networks:
      - web_network
networks:
  web_network:
    driver: bridge
```

Ejecutamos el comando `docker compose up -d` para crear el contenedor con nuestra imagen.

```
mlasmes@lubuntu-mlasmes:~/pedidos$ docker compose up -d
[+] Running 2/2
 ✔ Network pedidos_web_network  Created
 ✔ Container webserver          Started
mlasmes@lubuntu-mlasmes:~/pedidos$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED         STATUS
                                  NAMES
5262be242777   pedidos-web    "docker-php-entrypoi…"   3 seconds ago   Up 2 s
 0:80->80/tcp, [::]:80->80/tcp   webserver
```

La parada del servicio se hace con el comando `docker compose stop`, mientras que con `docker compose down` se paran y se eliminan los contenedores definidos en el fichero yaml. Por otra parte, si queremos copiar una nueva versión del proyecto a la imagen docker será preciso ejecutar `docker compose build web`, de forma que solo se construya la imagen con los cambios del servicio web mediante el Dockerfile.

```
mlasmes@lubuntu-mlasmes:~/pedidos$ docker compose stop
[+] Stopping 1/1
 ✔ Container webserver  Stopped
mlasmes@lubuntu-mlasmes:~/pedidos$ docker compose build web
[+] Building 1.2s (16/16) FINISHED
 => [internal] load local bake definitions
 => => reading from stdin 483B
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 1.30kB
 => WARN: LegacyKeyValueFormat: "ENV key=value" should be used instead o
 => [internal] load metadata for docker.io/library/php:8.4-apache
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [1/9] FROM docker.io/library/php:8.4-apache
 => [internal] load build context
 => => transferring context: 937.04kB
 => CACHED [2/9] RUN apt-get update && apt-get install -y     git     cu
 => CACHED [3/9] RUN docker-php-ext-install pdo pdo_mysql
 => CACHED [4/9] RUN a2enmod rewrite
 => CACHED [5/9] RUN sed -ri -e 's!/var/www/html!${APACHE_DOCUMENT_ROOT}
```

3. Implantación del servidor de BD

3.1. Despliegue del contenedor de BD con Docker Compose

El primer paso será elegir la imagen Docker para el contenedor de BD. Vamos a trabajar con la imagen del servidor de BD de código abierto [mariadb:10.11.2]

```
services:
  # Define el servicio web
  web:
    build: .
    container_name: webserver
    volumes:
      - .:/var/www/html
    ports:
      - "80:80"
    networks:
      - web_network
    restart: always
    depends_on:
      - db

  # Define el servicio de BD
  db:
    image: mariadb:10.11.2
    container_name: database
    environment:
      MARIADB_ROOT_PASSWORD: ${DB_ROOT_PASSWORD}
```

```
      MARIADB_DATABASE: ${DB_DATABASE}
      MARIADB_USER: ${DB_USER}
      MARIADB_PASSWORD: ${DB_PASSWORD}
    volumes:
      - database_data:/var/lib/mysql
    networks:
      - web_network
    restart: always


volumes:
  database_data:

networks:
  web_network:
    driver: bridge
```

Siguiendo la documentación de la imagen anterior, configura las variables de entorno en el fichero .env del proyecto especificando:

- ➔ El usuario de root: MARIADB_ROOT_PASSWORD
- ➔ El nombre de la BD: MARIADB_DATABASE
- ➔ Un usuario con acceso a la BD: MARIADB_USER
- ➔ La contraseña del usuario: MARIADB_PASSWORD.

Para que los datos de la BD persistan, es preciso crear un volumen (database_data) en el fichero compose.yaml mapeado, el cual se va a montar en el directorio indicado en la documentación de la imagen: `/var/lib/mysql`

No vamos a mapear en el host el puerto 3306 del contenedor mariadb, ya que la administración de la BD se hará abriendo una shell en el contenedor o accediendo por la red interna desde otro contenedor phpMyAdmin. Accede con una shell del contenedor a la BD vía CLI: docker exec -it <container_name> bash

Comprueba que se ha creado una BD con el nombre especificado en la variable de entorno y accede con el usuario vía CLI. En el caso de MariaDB accedemos con el usuario y listamos todas las bases de datos a las que tiene acceso, seleccionado la BD del proyecto, en la cual aún no hay ninguna tabla creada:

```
root@bfcda2aeb9b4:/# mariadb -u user_pedidos -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 3
Server version: 10.11.14-MariaDB-ubu2204 mariadb.org binary distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input state

MariaDB [(none)]> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| pedidos            |
+--------------------+
2 rows in set (0.001 sec)

MariaDB [(none)]> use pedidos;
Database changed
MariaDB [pedidos]> show tables;
Empty set (0.000 sec)

MariaDB [pedidos]>
```

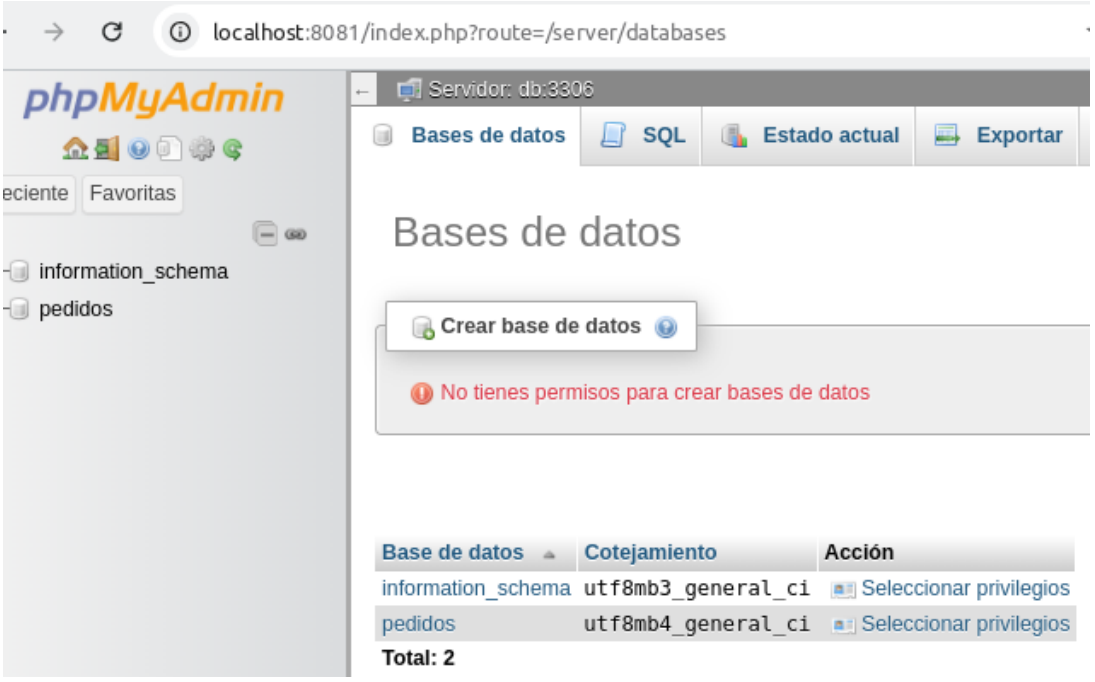
3.2. Contenedor para la gestión web del servidor de BD

En este apartado añadimos otro servicio al fichero de configuración de Docker Compose con la imagen phpmyadmin:

```
# Define el servicio de phpMyAdmin
 phpmyadmin:
   image: phpmyadmin
   container_name: phpmyadmin
   environment:
     PMA_HOST: db
     PMA_PORT: 3306
   ports:
     - 8081:80
   volumes:
     - phpmyadmin_data:/sessions
   networks:
     - web_network
   restart: always

volumes:
  phpmyadmin_data:
  database_data:
```

Accede vía web a phpMyAdmin (mapeando el puerto 8081 al 80 del contenedor) y comprueba que se ha creado la BD:



3.3. Instalación y configuración de Doctrine

En primer lugar, instala en el proyecto el paquete de Symfony que añade soporte para el ORM Doctrine, junto con un bundle que ayuda a generar código:

```
composer require symfony/orm-pack
composer require --dev symfony/maker-bundle
```

```
mlasmes@lubuntu-mlasmes:~/pedidos$ composer require symfony/orm-pack
./composer.json has been updated
Running composer update symfony/orm-pack
Loading composer repositories with package information
Updating dependencies
Lock file operations: 1 install, 0 updates, 0 removals
  - Locking symfony/orm-pack (v2.4.1)
Writing lock file
Installing dependencies from lock file (including require-dev)
Package operations: 1 install, 0 updates, 0 removals
  - Installing symfony/orm-pack (v2.4.1)
Generating autoload files
111 packages you are using are looking for funding.
Use the `composer fund` command to find out more!

Run composer recipes at any time to see the status of your Symfony recipes

  - Unpacked symfony/orm-pack
Executing script cache:clear [OK]
Executing script assets:install public [OK]
Executing script importmap:install [OK]
```

```
mlasmes@lubuntu-mlasmes:~/pedidos$ composer require --dev symfony/maker-bundle
./composer.json has been updated
Running composer update symfony/maker-bundle
Loading composer repositories with package information
Updating dependencies
Nothing to modify in lock file
Writing lock file
Installing dependencies from lock file (including require-dev)
Nothing to install, update or remove
Generating autoload files
110 packages you are using are looking for funding.
Use the `composer fund` command to find out more!

Run composer recipes at any time to see the status of your Symfony recipes.

Executing script cache:clear [OK]
Executing script assets:install public [OK]
Executing script importmap:install [OK]

No security vulnerability advisories found.
Using version ^1.64 for symfony/maker-bundle
mlasmes@lubuntu-mlasmes:~/pedidos$
```

La información de la conexión a la BD se almacena en una variable de entorno llamada `DATABASE_URL`, cuyo valor se establece en el fichero de configuración .env. Ten en cuenta que se debe indicar el nombre del servicio db en lugar de 127.0.0.1 o localhost, ya que la BD corre en otro contenedor. Los contenedores se comunican a través de una red interna y resuelven el nombre del servicio que se indica en compose.yaml por la IP interna que tenga el contenedor.

DATABASE_URL=mysql://db_user:db_password@db:3306/db_name

Del mismo modo, se puede especificar la versión de MySQL añadiendo a la URL el caracter ? seguido del parámetro serverVersion. Se pueden configurar más opciones en el fichero config/packages/doctrine.yaml.

```
DATABASE_URL="mysql://db_user:db_password@db:3306/db_name?serverVersion=10.
11.2-MariaDB&charset=utf8mb4"
```

Una vez que se han configurado los parámetros de la conexión a la BD en el fichero .env, será preciso eliminar el contenedor web, volver a construir la imagen y crearlo de nuevo a partir de la nueva imagen de forma que incluya los cambios en los ficheros del proyecto. Para ello, ejecutamos lo siguiente:

```
docker compose down web
docker compose build web
docker compose up -d
```

Desde Doctrine se puede intentar crear la BD mediante el siguiente comando ejecutado desde la shell del contenedor web, que crea el fichero .php en `src/Entity/` con la definición de la clase de la entidad:

```
php bin/console doctrine:database:create
```

En nuestro caso se obtiene un error porque ya existe una BD vacía con el mismo nombre creada por Docker Compose según el valor establecido en la variable de entorno `DB_DATABASE` del fichero .env. No obstante, este comando nos permite comprobar que el ORM Doctrine es capaz de conectar el servidor de BD que corre en el contenedor `db`.

```
mlasmes@lubuntu-mlasmes:~/pedidos$ docker exec -it webserver bash
root@ff664cc505a7:/var/www/html# php bin/console doctrine:database:create
Could not create database "pedidos" for connection named default
An exception occurred while executing a query: SQLSTATE[42P04]: Duplicate database: 7 ERRO
R:  database "pedidos" already exists
root@ff664cc505a7:/var/www/html#
```

4. Seguridad: Usuarios y roles

4.1. Requisitos previos

En el directorio del proyecto public se tienen todos los ficheros estáticos de la aplicación (imágenes, hojas de estilo, JS) y el fichero index.php que implementa el controlador frontal. Se trata del directorio raíz del servidor web Apache, cuya ruta se especifica en la directiva DocumentRoot.

Según el siguiente artículo del manual, para que se traduzcan las rutas en Apache es necesario utilizar el módulo rewrite de Apache, de forma que todas las peticiones se envíen al controlador frontal, el cual a su vez ejecutará el controlador correspondiente según la ruta especificada.

En un entorno de prueba se pueden configurar las reglas del módulo rewrite mediante un fichero .htaccess en el directorio public del proyecto. Para ello, se puede instalar el bundle apache-pack (**`composer require symfony/apache-pack`**), el cual genera el fichero .htaccess necesario en public. Comprueba el contenido del fichero .htaccess generado.

En el caso de un entorno en producción se recomienda implementar estas reglas del módulo rewrite dentro de la configuración del virtual host o en el fichero de configuración global de Apache, debido a que la configuración a nivel de htaccess no se recomienda porque afecta al rendimiento, ya que el servidor web tiene que leer el fichero .htaccess en cada petición que se haga a un fichero del directorio public.

Para que Apache no ignore el fichero .htaccess, es preciso configurar la directiva `AllowOverride` a `All` en lugar de `None`. Este cambio se puede hacer a nivel global en el fichero /etc/apache2/apache2.conf o a nivel de un virtual host concreto en el fichero de configuración del sitio web en /etc/apache2/sites-available.

`Modificamos /etc/apache2/apache2.conf`

`AllowOverride None → All`

Añadimos este cambio en el fichero /etc/apache2/apache2.conf en la imagen modificando el Dockerfile para añadir la siguiente línea:

```
COPY web/apache2.conf /etc/apache2/apache2.conf
```

Tras esto, volvemos a construir la imagen y a crear un nuevo contenedor web a partir de la nueva versión de la imagen:

```
docker compose down web
docker compose build web
docker compose up -d
```

Comprueba mediante una shell al contenedor que se haya modificado el fichero /etc/apache2/apache2.conf

4.2. Creación de la entidad Usuario

La entidad Usuario se va a integrar en el componente de seguridad de Symfony (tiene que implementar UserInterface). Según se describe en la documentación del bundle Security de Symfony, para crear la clase de la entidad Usuario se puede ejecutar el siguiente comando: **`php bin/console make:user`**

Como resultado, se obtiene el fichero `src/Entity/Usuario.php` y también se añaden cambios de configuración en el fichero `config/packages/security.yaml`. Comprueba su contenido y localiza la propiedad que sirve de identificador de usuario.

Una vez generada la clase Usuario, creamos la tabla correspondiente en la BD mediante el ORM doctrine ejecutando lo siguiente en la shell del contenedor web:

```
php bin/console make:migration
php bin/console doctrine:migrations:migrate
```

Muestra la estructura de la tabla que se ha creado en la base de datos mediante comandos (`show create table Usuario`, `describe Usuario`, `\d+ Usuario`) o vía web mediante phpMyAdmin o Dbeaver.

Por último, es posible añadir nuevas propiedades a la entidad Usuario, para lo cual ejecutamos el comando `php bin/console make:entity Usuario` indicando la entidad Usuario ya existente, de forma que se crea el atributo de la clase junto con sus getters y setters.

4.3. Registro de usuarios

El bundle Security proporciona las funciones de hashing de contraseñas y verificaciones de hashes. Para ello, es preciso que la clase Usuario implemente la interfaz PasswordAuthenticatedUserInterface. El comando anterior make:user ya se encargó de configurar el fichero security.yaml para que use el servicio hasher de contraseñas.

Por un lado, es posible obtener manualmente el hash a partir de una contraseña en texto claro: `php bin/console security:hash-password`. Añade algún usuario insertando los datos en la BD, junto con el hash de la contraseña.

Por otro lado, se puede crear un controlador y un formulario de registro de usuarios que calcule el hash de la contraseña automáticamente para guardarlo en la base de datos.

```
composer require symfonycasts/verify-email-bundle
php bin/console make:registration-form
```

Implementa la verificación del email introducido por el usuario antes de guardarlo usando para ello el servicio SymfonyCastsVerifyEmailBundle.

4.4. Formulario de login

El primer formulario que vamos a crear es el de login. Existe un comando para crear de forma automática todo lo necesario para implementar un formulario de login (plantilla, controlador y configuración necesaria en el fichero security.yaml):

```
php bin/console make:security:form-login
```

Comprueba que se ha creado la plantilla correspondiente en el directorio templates/login con el formulario HTML. El atributo action del elemento <form> debe apuntar a la ruta de login:

```
<form action="{{ path('login') }}" method="post">
         <label for="username">Email:</label>
         <input type="text" id="username" name="_username" value="{{
      last_username }}"/>
         <label for="password">Password:</label>
          <input type="password" id="password" name="_password"/>
          <button type="submit">login</button>

 </form>
```

4.5. Habilitar FormLoginAuthenticator

Comprueba en el fichero `security.yaml` que esté habilitado el uso de FormLoginAuthenticator mediante la directiva **form_login** en el firewall main. Las peticiones hechas a una ruta securizada por un usuario no logado se reenvían a la ruta login_path. Tras el login, se reenvía a la URL que se solicitó, la cual se guardó en la sesión. En el login_path se indica la ruta a la que se reenvía a un usuario no autenticado si no solicitó otra URL distinta al login. check_path (ruta a la que se envía el formulario de login), default_target_path (ruta por defecto tras hacer login con éxito si no se accedió a ninguna página previa en la sesión). La opción always_use_default_target_path a true indica que se ignore la URL previa a la hora de redirigir al usuario, redirigiendo siempre a la página por defecto.

```
php bin/console make:security:form-login


security:
    firewalls:
        main:
            form_login:
            # "login" is the name of the route created previously
                  login_path: login
                        check_path: login
                        default_target_path: ruta_defecto_tras_login
                        always_use_default_target_path: true
```

4.6. Control de acceso

Se puede securizar el acceso a los controladores mediante el atributo #[IsGranted], a nivel de la clase controladora o en un método controlador. En nuestro caso, añadimos el atributo a la clase BaseController para que se requiera el ROLE_USER que por defecto tienen todos los usuarios que hayan abierto sesión.

```
use Symfony\Component\Security\Http\Attribute\IsGranted;

#[IsGranted('ROLE_USER')]
```

Es preciso restringir el acceso a las rutas ^/admin (que empiezan por /admin) para que sólo puedan acceder los usuarios con ROLE_ADMIN.

En el fichero `security.yalm`

```
        - { path: ^/admin, roles: ROLE_ADMIN }
```

https://symfony.com/doc/current/the-fast-track/en/15-security.html

5. Creación de un CRUD

5.1. Dashboard de administración

Vamos a usar el bundle EasyAdmin para automatizar y estandarizar la generación del código de los CRUD y disponer de un backend de administración (ruta "/admin") con enlaces a los controladores CRUD.

Según la documentación, una vez instalado el bundle (`composer require easycorp/easyadmin-bundle`), el primer paso consiste en crear los dashboards de los usuarios que contendrán un menú con los enlaces necesarios para acceder a los CRUDs. Para ello, ejecutamos lo siguiente: **`php bin/console make:admin:dashboard`**

Si accedemos a la ruta /admin con el navegador se mostrará la página de bienvenida por defecto de EasyAdmin. Comprueba que /admin aparece disponible dentro de las rutas del proyecto mediante el comando **`php bin/console debug:router`**

El maker anterior crea una clase controladora en la ubicación src/Controller/Admin/DashboardController.php para el dashboard, la cual deberá extender de la clase AbstractDashboardController o implementar la interfaz DashboardControllerInterface. Incluye los métodos index(), configureDashboard() y configureMenuItems(), que van a servir para personalizar el backend de administración.

- La ruta del dashboard puede establecerse mediante el atributo `#[AdminDashboard]` de la clase.

  ```
  #[AdminDashboard(routePath: '/admin', routeName: 'admin')]

  class DashboardController extends AbstractDashboardController

  {

      // ...
  ```

- La configuración del dashboard se define mediante el método **configureDashboard**(). Se pueden cambiar aspectos del dashboard como el título.

  ```
  return Dashboard::new()->setTitle('Tienda de Papel Admin')
  ```

  Alternativamente, en lugar de mostrar la página por defecto del dashboard,

se puede redirigir directamente hacia la URL de una de las tareas de administración más comunes. Así, el CRUD de una de las entidades sería la página de inicio del usuario administrador. Para ello, en el controlador index() generamos y redirigimos a la URL de uno de los CRUDs:

```
use EasyCorp\Bundle\EasyAdminBundle\Router\AdminUrlGenerator;
…


return
$this->redirect($adminUrlGenerator->setController(OneOfYourCrudContro
ller::class)->generateUrl());
```

También se puede redireccionar a una página diferente dependiendo del usuario:

```
if ('dgarama' === $this->getUser()->getUsername()) {

            return $this->redirect('...');

}
```

O hacer return $this->render de una plantilla, por ejemplo una que extienda de @EasyAdmin/layaout.html.twig. Los servicios se deben inyectar en el constructor de la clase controladora en lugar de en el método index(), ya que index() es parte de la interfaz Dashboard().

```
public function index(): Response
{
  $chart = $this->chartBuilder->createChart(Chart::TYPE_LINE);
  // ...set chart data and options somehow
  return $this->render('admin/my-dashboard.html.twig', [
         'chart' => $chart,
  ]);



{# templates/admin/my_dashboard.html.twig #}
{% extends '@EasyAdmin/layout.html.twig' %}

{% block main %}
    {# ... #}
{% endblock main %}
```

- Mediante el método **configureMenuItems**() añadimos en el menú principal del panel de control los enlaces a los controladores de los CRUDs, asociando así los recursos al dashboard. Por seguridad, solo se podrá acceder a los CRUDs que estén asociados al menú principal.

  Podríamos añadir un link a la página principal del proyecto y links al resto de los CRUD. El menú principal consiste en una colección de objetos que implementan la interfaz EasyCorp\Bundle\EasyAdminBundle\Contracts\Menu\MenuInterface para configurar el comportamiento y apariencia de los elementos del menú.

```
#[AdminDashboard(routePath: '/admin', routeName: 'admin')]
class DashboardController extends AbstractDashboardController
{
    // ...

  public function configureMenuItems(): iterable
  {
```

```
    return [
    MenuItem::linkToDashboard('Dashboard', 'fa fa-home'),
    MenuItem::section('Blog'),
    MenuItem::linkToCrud('Categories', 'fa fa-tags', Category::class),
    MenuItem::linkToCrud('Blog Posts', 'fa fa-file-text', BlogPost::class),
    MenuItem::section('Users'),
    MenuItem::linkToCrud('Comments', 'fa fa-comment', Comment::class),
    MenuItem::linkToCrud('Users', 'fa fa-user', User::class),
    ];
   }
}
```

5.2. Controladores CRUD

En Symfony se puede crear un controlador CRUD (Create, Read, Update y Delete) para una entidad del ORM Doctrine mediante el comando **`php bin/console make:admin:crud`**. Se trata de una clase controladora que hereda de AbstractCrudController o que implementa la interfaz CrudControllerInterface, asegurando de este modo que se implementen una serie de métodos. Cada controlador CRUD puede estar asociado con uno o varios dashboards.

Para cada una de las acciones del controlador se genera una ruta. Por ejemplo, para un controlador llamado ProductCrudController que pertenece a un backend de administración con ruta /admin, se generarían las siguientes rutas:

Admin route name	Admin route path
`admin_product_index`	`/admin/product`
`admin_product_new`	`/admin/product/new`
`admin_product_batch_delete`	`/admin/product/batch-delete`
`admin_product_autocomplete`	`/admin/product/autocomplete`
`admin_product_edit`	`/admin/product/324/edit`
`admin_product_delete`	`/admin/product/324/delete`
`admin_product_detail`	`/admin/product/324`

Los nombres de las rutas se pueden personalizar mediante la opción `routes` del atributo `#[AdminDashboard]` de la clase. Por ejemplo, las rutas anteriores para el CRUD de la entidad Product podrían modificarse del siguiente modo:

```
use EasyCorp\Bundle\EasyAdminBundle\Attribute\AdminDashboard;
// ...
```

```
#[AdminDashboard(routePath: '/admin', routeName: 'admin', routes: [
    'index' => ['routePath' => '/all'],
    'new' => ['routePath' => '/create', 'routeName' => 'create'],
       'edit'  =>  ['routePath'  =>  '/editing-{entityId}',  'routeName'  =>
'editing'],
    'delete' => ['routePath' => '/remove/{entityId}'],
    'detail' => ['routeName' => 'view'],
])]
class SomeDashboardController extends AbstractDashboardController
{
    // ...
}
```

Admin route name	Admin route path
`admin_product_index`	`/admin/product/all`
`admin_product_create`	`/admin/product/create`
`admin_product_batch_delete`	`/admin/product/current/batch-delete`
`admin_product_autocomplete`	`/admin/product/current/autocomplete`
`admin_product_editing`	`/admin/product/current/editing-324`
`admin_product_delete`	`/admin/product/remove/324`
`admin_product_view`	`/admin/product/324`

De forma alternativa, el nombre de la ruta se puede también modificar específicamente en el controlador de cada acción mediante el atributo `#[AdminRoute]`.

```
use EasyCorp\Bundle\EasyAdminBundle\Attribute\AdminRoute;
// ...

class ProductCrudController extends AbstractCrudController
{
    // ...

    #[AdminRoute(path: '/latest-products', name: 'latest')]
    public function index(AdminContext $context)
    {
        // ...
    }

}
```

Una vez se han definido las rutas de cada una de las acciones del CRUD, es preciso configurar la entidad que va a manejar el CRUD. Para ello, se define el método estático y público getEntityFqcn() para que devuelva el nombre de la clase de la entidad:

```
public static function getEntityFqcn(): string
    {
        return Product::class;
    }
```

El resto de opciones no son obligatorias y se pueden configurar mediante el método **configureCrud**():

```
class ProductCrudController extends AbstractCrudController
{
    // ...

    public function configureCrud(Crud $crud): Crud
    {
        return $crud
            ->setEntityLabelInSingular('...')
            ->setDateFormat('...')
            // ...
        ;
    }
```

5.3. Visualización de campos de los CRUDs

En cada página de un CRUD se mostrarán una serie de campos con los contenidos de las propiedades de la entidad. Aunque se tiene una configuración por defecto para la visualización de los campos en la página del CRUD, la vista se puede personalizar implementando el método **configureFields**(), el cual permite hacer cambios como modificar los títulos de las columnas, quitar propiedades de la entidad que no se deban editar (id por ejemplo), etc.

Configuramos los campos a mostrar indicando como parámetro el nombre de cada propiedad de la entidad:

```
use EasyCorp\Bundle\EasyAdminBundle\Field\Field;

public function configureFields(string $pageName): iterable
{
    return [
        Field::new('title'),
        Field::new('description'),
        Field::new('price'),
        Field::new('stock'),
        Field::new('publishedAt'),
    ];
}
```

Se pueden mostrar también campos que no existan como columna en la tabla pero que se pueden obtener a partir de las propiedades de la entidad:

```
public function configureFields(string $pageName): iterable
{
    return [
        TextField::new('fullName'),
        // ...
    ];
}
```

En este ejemplo, sería necesario añadir el getter correspondiente en la clase de la entidad:

```
use Doctrine\ORM\Mapping as ORM;

#[ORM\Entity]
class Customer
{
    // ...

    public function getFullName()
    {
        return $this->getFirstName().' '.$this->getLastName();
    }
}
```

5.4. Filtros en los CRUDs

Por último, es posible añadir filtros en la página del CRUD que permiten listar registros seleccionándolos según los valores de las propiedades de la entidad. Esto se consigue implementando el método **configureFilters**() en la clase que hereda de AbstractCrudController.

```
namespace App\Controller\Admin;

use EasyCorp\Bundle\EasyAdminBundle\Config\Filters;
use
EasyCorp\Bundle\EasyAdminBundle\Controller\AbstractCrudController;

class ProductCrudController extends AbstractCrudController
{
    // ...

    public function configureFilters(Filters $filters): Filters
    {
        return $filters
            ->add('title')
            ->add('price')
            ->add('published')
        ;
    }
}
```

5.5. Hashing de la contraseña en el CRUD de usuarios

Por defecto, el CRUD de usuarios no calcula el hash de la contraseña introducida en texto claro. Para solventarlo, vamos a sobrescribir los métodos de la clase AbstractCrudController (`createEntity()`, `updateEntity()`, `persistEntity()` and `deleteEntity()`) en la clase hija UserCrudController (controladores del CRUD de la entidad User), de forma que se puedan añadir instrucciones adicionales que permitan calcular el hash bcrypt.

```
use Symfony\Component\Form\Extension\Core\Type\PasswordType;
use Symfony\Component\PasswordHasher\Hasher\UserPasswordHasherInterface;
use EasyCorp\Bundle\EasyAdminBundle\Event\BeforeEntityPersistedEvent;

class UsuarioCrudController extends AbstractCrudController
{
    /**
     * @var UserPasswordHasherInterface
     */
    private $userPasswordHasher;
```

```
    /**
     * Inicializamos el PasswordHasher
     * @param UserPasswordHasherInterface $userPasswordHasher
     */
               public    function    __construct(UserPasswordHasherInterface
$userPasswordHasher)
    {
        $this->userPasswordHasher = $userPasswordHasher;
    }

      public function updateEntity($entity)
          {
              $this->hashPassWord($entity);
              parent::updateEntity($entity);
          }

      public function persistEntity($entity)
          {
              this->hashPassWor$d($entity);
              parent::persistEntity($entity);
          }

      public function hashPassWord(BeforeEntityPersistedEvent $event)
      {
          $entity = $event->getEntityInstance();


      // comprobamos si se trata de una entidad User para continuar
          if (!($entity instanceof User)) {
              return;
          }


      $entity->setPassword($this->userPasswordHasher->hashPassword($entity,
      $entity->getPlainPassword()));
      }
```

Para guardar la contraseña en texto claro, creamos una propiedad adicional en la entidad User, la cual no se va a persistir en la BD.

Do not set to change the $password field directly. Use $plainPassword and if it is set, then encode $password

```
      // ...
      class User implements UserInterface
      {
          //...

         /**
           * @var string The hashed password
           * @ORM\Column(type="string")
           */
          private $password;

          /**
           * @var string
           */
          private $plainPassword;
```

```
    /**
     * @see UserInterface
     */
    public function eraseCredentials()
    {
          // If you store any temporary, sensitive data on the user,
clear it here
         $this->plainPassword = null;
    }

 public function getPlainPassword(){
       $this->plainPassword = $this->getPassword();
       return $this->plainPassword;
    }

}
```

5.6. Restringir el acceso al backend de administración

Todas las rutas que pertenecen al mismo panel de control comparten un mismo prefijo (/admin, /dashboard, etc), según se haya configurado en el DashboardController. Esto permite configurar fácilmente el control de acceso en base a los roles mediante la opción access_control del fichero config/packages/security.yaml.

```
# config/packages/security.yaml
security:
    # ...

    access_control:
        # change '/admin' to the prefix used by your Dashboard URLs
        - { path: ^/admin, roles: ROLE_ADMIN }
```

Alternativamente, se puede usar el atributo #[IsGranted] con el inconveniente de que habría que establecerlo en todos los controladores de los CRUDs y en el controlador del dashboard.

```
#[AdminDashboard(routePath: '/admin', routeName: 'admin')]
#[IsGranted('ROLE_ADMIN')]
class DashboardController extends AbstractDashboardController
{
    // ...
}
```